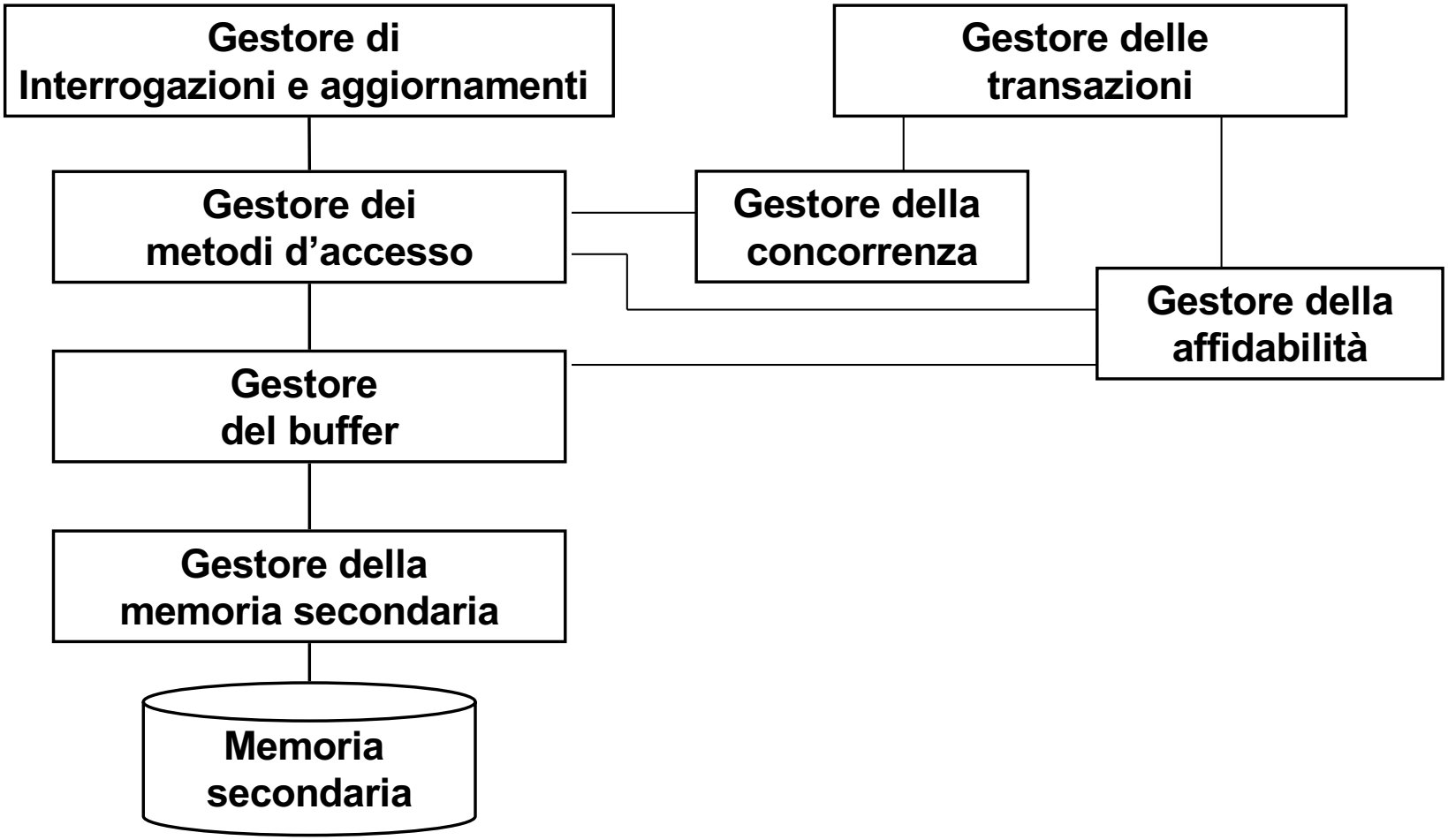
Gestione della concorrenza

Gestore degli accessi e delle interrogazioni

Gestore delle transazioni



Controllo di concorrenza

- La concorrenza è fondamentale:
  - decine o centinaia di transazioni al secondo
    - banche, prenotazioni aeree, social network
  - non possono essere seriali (ognuna per intero, senza azioni intercalate)

Problema

- Anomalie causate dall'esecuzione concorrente, che quindi va governata

Due prelevamenti da un conto bancario

- Quanto c'è sul conto?
  - 1000
- Quanto c'è sul conto?
  - 1000
- Bene, allora prelevo 200
  - Ok, il nuovo saldo è 800
- Bene, allora prelevo 100
  - Ok, il nuovo saldo è 900

Perdita di aggiornamento (lost update)

- t_1 : r(x), $x = x - 200$, w(x)
- t_2 : r(x), $x = x - 100$, w(x)

- Inizialmente $x = 1000$; dopo esecuzione seriale (tutta t_1 e poi tutta t_2) $x = 700$
- Abbiamo visto un'esecuzione concorrente:

t_1	t_2
$r_1(x)$	
$x = x - 200$	
	$r_2(x)$
	$x = x - 100$
$w_1(x)$	
`commit`	
	$w_2(x)$
	`commit`

- Un aggiornamento viene perso: alla fine x=900
- Le azioni delle transazioni si alternano (Paperina, Paperino, Paperina, …) in modo inaccettabile
- Se eseguissimo in modo seriale (Paperina, Paperina, Paperino, ...) allora ok

Lettura prima della conferma (commit)

1000 sul conto

- Ecco un assegno da 10.000
  - Bene, il nuovo saldo potrebbe essere 11.000
  - Ma l'assegno è falso!
  - Annullo l'operazione
  - Il saldo resta 1000

- Quanto c'è sul conto?
  - 11.000
- Abbiamo 11.000, evviva!!
- Paperina ha letto un dato sbagliato!

Lettura sporca (dirty read)

t_1	t_2
$r_1(x)$	
$x = x + 11000$	
$w_1(x)$	
	$r_2(x)$
`abort`	

- t_2 ha letto uno stato intermedio ("sporco") e lo può comunicare all'esterno

Lettura, scrittura, lettura

1000 sul conto A
1000 sul conto B

- Quanto c'è sul conto A?
  - 1000
- Sposta 500 da A a B
  - Nuovo saldo A: 500
  - Nuovo saldo B: 1500
- Quanto c'è sul conto B?
  - 1500
- Ehi, abbiamo 2500!

Aggiornamento fantasma (ghost update), lettura inconsistente (con la base di dati)

- y=1000 e z= 1000
- quindi $y + z = 2000$

t_1	t_2
$r_1(y)$	
	$r_2(y)$
	$y = y - 500$
	$r_2(z)$
	$z = z + 500$
	$w_2(y)$
	$w_2(z)$
	`commit`
$r_1(z)$	
$s = y + z$	

- $s = 2500$: ma la somma $y + z$ non è mai stata "davvero" 2500:
  - t_1 vede uno stato non esistente (o non coerente)

Letture inconsistenti (inconsistent read), caso ancora più semplice

- t_1 legge due volte x :

t_1	t_2
$r_1(x)$	
	$r_2(x)$
	$x = x + 1$
	$w_2(x)$
	`commit`
$r_1(x)$	

- t_1 legge due valori diversi per x !
- Esempio:
  - un join con nested-loop, si legge più volte una stessa relazione

Inserimento fantasma (phantom)

- Esempio: sistema di prenotazione

t_1	t_2
"conta i posti disponibili"	
	"aggiungi nuovi posti" (aereo più grande)
	`commit`
"conta i posti disponibili"	

Anomalie

- **Perdita di aggiornamento** **W-W**
- **Lettura sporca** **R-W (o W-W) con abort**
- **Letture inconsistenti** **R-W**
- **Aggiornamento fantasma** **R-W**
- **Inserimento fantasma** **R-W su dato "nuovo"**

Livelli di isolamento in SQL:1999 (e JDBC)

- Le transazioni possono essere definite **`read-only`** (non possono scrivere)
- Il livello di isolamento può essere scelto per ogni transazione
  - **`read uncommitted`** permette letture sporche, letture inconsistenti (e aggiornamenti fantasma) e inserimenti fantasma
  - **`read committed`** evita letture sporche ma permette letture inconsistenti (e aggiornamenti fantasma) e inserimenti fantasma
  - **`repeatable read`** evita tutte le anomalie esclusi gli inserimenti fantasma
  - **`serializable`** evita tutte le anomalie
- Nota:
  - la perdita di aggiornamento <u>dovrebbe</u> essere sempre evitata, <u>ma non è così</u>: è necessario usare sempre **`serializable`** per le transazioni che scrivono

Anomalie

- `read uncommitted`

Lettura sporca	R-W (o W-W) con abort

- `read committed`

Letture inconsistenti	R-W
Aggiornamento fantasma	R-W

- `repeatable read`

Inserimento fantasma	R-W su dato "nuovo"

- `serializable`

- Ogni livello di isolamento evita le anomalie che lo precedono nell'elenco e permette quelle che lo seguono

Livelli di isolamento, perché?

- La gestione del controllo della concorrenza è costosa, se non serve, possiamo rinunciarci
- Nota bene: per le letture, non per le scritture

Livelli di isolamento, esempi

- Vedi compito d'esame del 22/09/2017 domanda 5

- Clienti(<u>Codice</u>,Negozio,Punti)

```
SELECT Negozio,
        count(*) as numClienti,
        sum(Punti) as totalePunti,
        sum(Punti)/ count(*) as mediaPunti
FROM Clienti
GROUP BY Negozio
```

Domanda 5 (20%)
Una catena di supermercati ha una base di dati dei propri clienti che dispongono di una "tessera fedeltà," con varie informazioni su ciascun cliente, fra cui (a) il totale dei punti acquisiti attraverso l'uso della tessera e (b) il negozio della catena cui fa riferimento (ad esempio, quello presso cui ha inizialmente richiesto la tessera). Si vuole eseguire su di essa l'interrogazione che calcola, per ciascun negozio, il numero dei clienti, la somma dei punti fedeltà dei clienti e la relativa media per cliente. Indicare quale livello di isolamento (`READ UNCOMMITTED`, `READ COMMITTED`, `REPEATABLE READ` o `SERIALIZABLE`) si potrebbe scegliere in ciascuno dei seguenti casi (si supponga che, in generale, sia stato rilevato che, nel corso degli inserimenti e delle modifiche, vengono inseriti valori sbagliati anche di vari ordini di grandezza, che sono poi corretti prima del commit):

1. L'operazione è eseguita mentre vengono inseriti alcuni nuovi clienti (per ciascun negozio pochi rispetto a quelli già presenti), con la finalità di acquisire informazioni approssimate ma ragionevolmente indicative sugli andamenti complessivi.
2. L'operazione è eseguita mentre vengono modificati i valori dei punti fedeltà di tutti i clienti (a seguito di una ridefinizione dei criteri di assegnazione dei punti stessi), con la finalità di acquisire informazioni approssimate ma ragionevolmente indicative sugli andamenti complessivi.
3. L'operazione è eseguita in un momento in cui non ci sono aggiornamenti di alcun genere, con la finalità di individuare i primi tre negozi da premiare in una campagna promozionale sulla base dei punti acquisiti dai rispettivi clienti.
4. L'operazione è eseguita mentre vengono inseriti alcuni nuovi clienti (per ciascun negozio pochi rispetto a quelli già presenti), con la finalità di individuare i primi tre negozi da premiare in una campagna promozionale sulla base dei punti acquisiti dai rispettivi clienti.
5. L'operazione è eseguita mentre vengono modificati i valori dei punti fedeltà di tutti i clienti (a seguito di una ridefinizione dei criteri di assegnazione dei punti stessi), con la finalità di individuare i primi tre negozi da premiare in una campagna promozionale sulla base dei punti acquisiti dai rispettivi clienti.

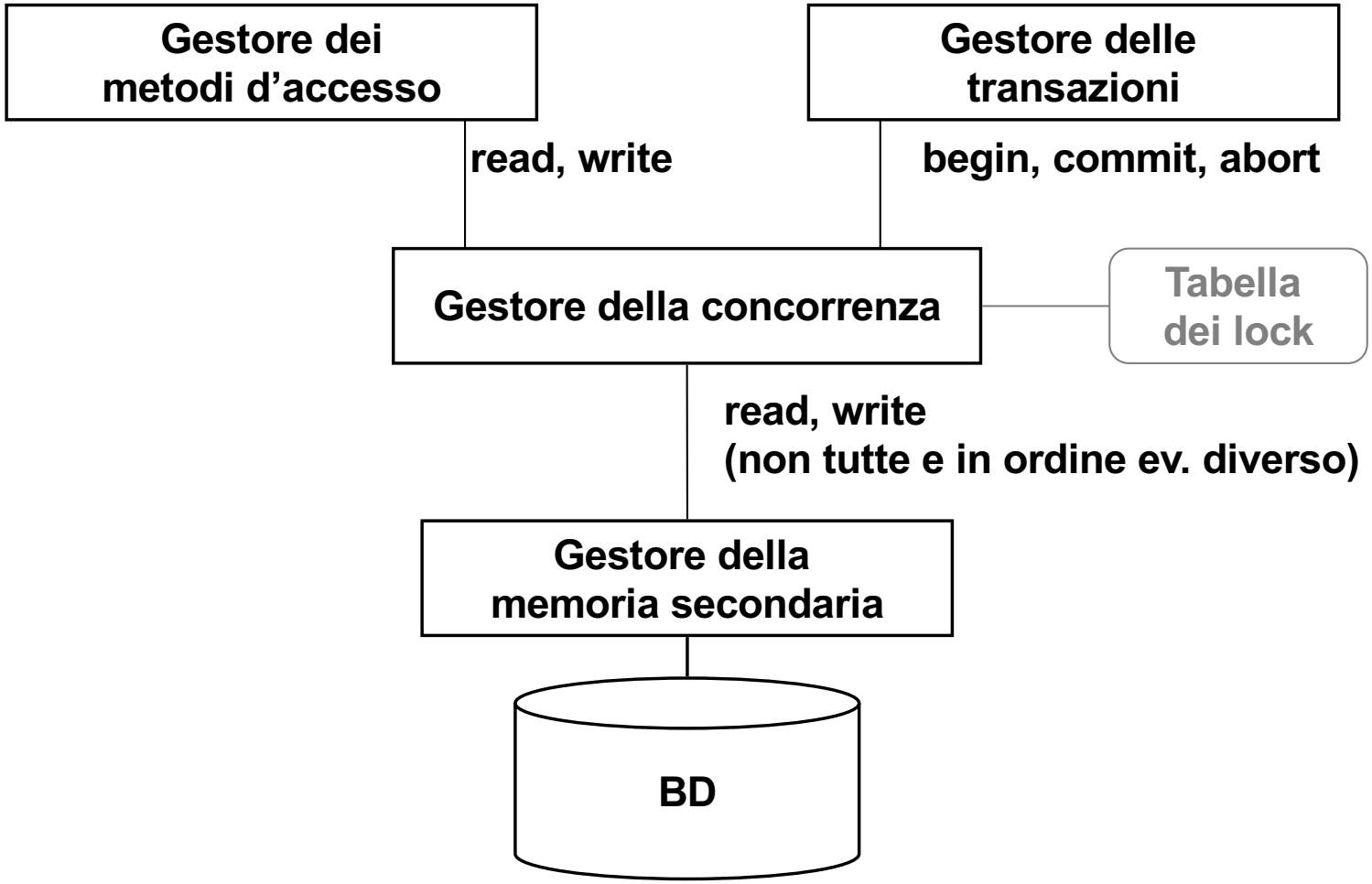
1.	2.	3.	4.	5.

Domanda 5 (20%)
Una catena di supermercati ha una base di dati dei propri clienti che dispongono di una "tessera fedeltà," con varie informazioni su ciascun cliente, fra cui (a) il totale dei punti acquisiti attraverso l'uso della tessera e (b) il negozio della catena cui fa riferimento (ad esempio, quello presso cui ha inizialmente richiesto la tessera). Si vuole eseguire su di essa l'interrogazione che calcola, per ciascun negozio, il numero dei clienti, la somma dei punti fedeltà dei clienti e la relativa media per cliente. Indicare quale livello di isolamento (`READ UNCOMMITTED`, `READ COMMITTED`, `REPEATABLE READ` o `SERIALIZABLE`) si potrebbe scegliere in ciascuno dei seguenti casi (si supponga che, in generale, sia stato rilevato che, nel corso degli inserimenti e delle modifiche, vengono inseriti valori sbagliati anche di vari ordini di grandezza, che sono poi corretti prima del commit):

1. L'operazione è eseguita mentre vengono inseriti alcuni nuovi clienti (per ciascun negozio pochi rispetto a quelli già presenti), con la finalità di acquisire informazioni approssimate ma ragionevolmente indicative sugli andamenti complessivi.
2. L'operazione è eseguita mentre vengono modificati i valori dei punti fedeltà di tutti i clienti (a seguito di una ridefinizione dei criteri di assegnazione dei punti stessi), con la finalità di acquisire informazioni approssimate ma ragionevolmente indicative sugli andamenti complessivi.
3. L'operazione è eseguita in un momento in cui non ci sono aggiornamenti di alcun genere, con la finalità di individuare i primi tre negozi da premiare in una campagna promozionale sulla base dei punti acquisiti dai rispettivi clienti.
4. L'operazione è eseguita mentre vengono inseriti alcuni nuovi clienti (per ciascun negozio pochi rispetto a quelli già presenti), con la finalità di individuare i primi tre negozi da premiare in una campagna promozionale sulla base dei punti acquisiti dai rispettivi clienti.
5. L'operazione è eseguita mentre vengono modificati i valori dei punti fedeltà di tutti i clienti (a seguito di una ridefinizione dei criteri di assegnazione dei punti stessi), con la finalità di individuare i primi tre negozi da premiare in una campagna promozionale sulla base dei punti acquisiti dai rispettivi clienti.

1.	2.	3.	4.	5.
Read committed	Repetable read	Read uncommitted	Serializable	Repetable read

Gestore della concorrenza (ignorando buffer e affidabilità)



“Teoria” del controllo di concorrenza

- Transazione: sequenza di letture e scritture (senza ulteriore semantica) e conclusione (commit o abort)
- Notazione:
  - ogni transazione ha un identificatore univoco
- Esempio:
  - $t_1 : r_1(x)\ r_1(y)\ w_1(x)\ w_1(y)\ \mathrm{c}_1$

Schedule

- Sequenza di operazioni di input/output
- Ipotesi semplificativa preliminare (non accettabile in pratica, la rimuoveremo):
  - ignoriamo le transazioni che vanno in abort (e quindi non scriviamo commit e abort)
- Esempio:

$$S : r_1(x)\ r_2(z)\ w_1(x)\ w_2(z)$$

schedule per le transazioni

$$t_1 : r_1(x)\ w_1(x)$$
$$t_2 : r_2(z)\ w_2(z)$$

- Nota, quindi consideriamo la **commit-proiezione** degli schedule reali, in quanto ignoriamo le transazioni che vanno in abort, rimuovendo tutte le loro azioni dallo schedule

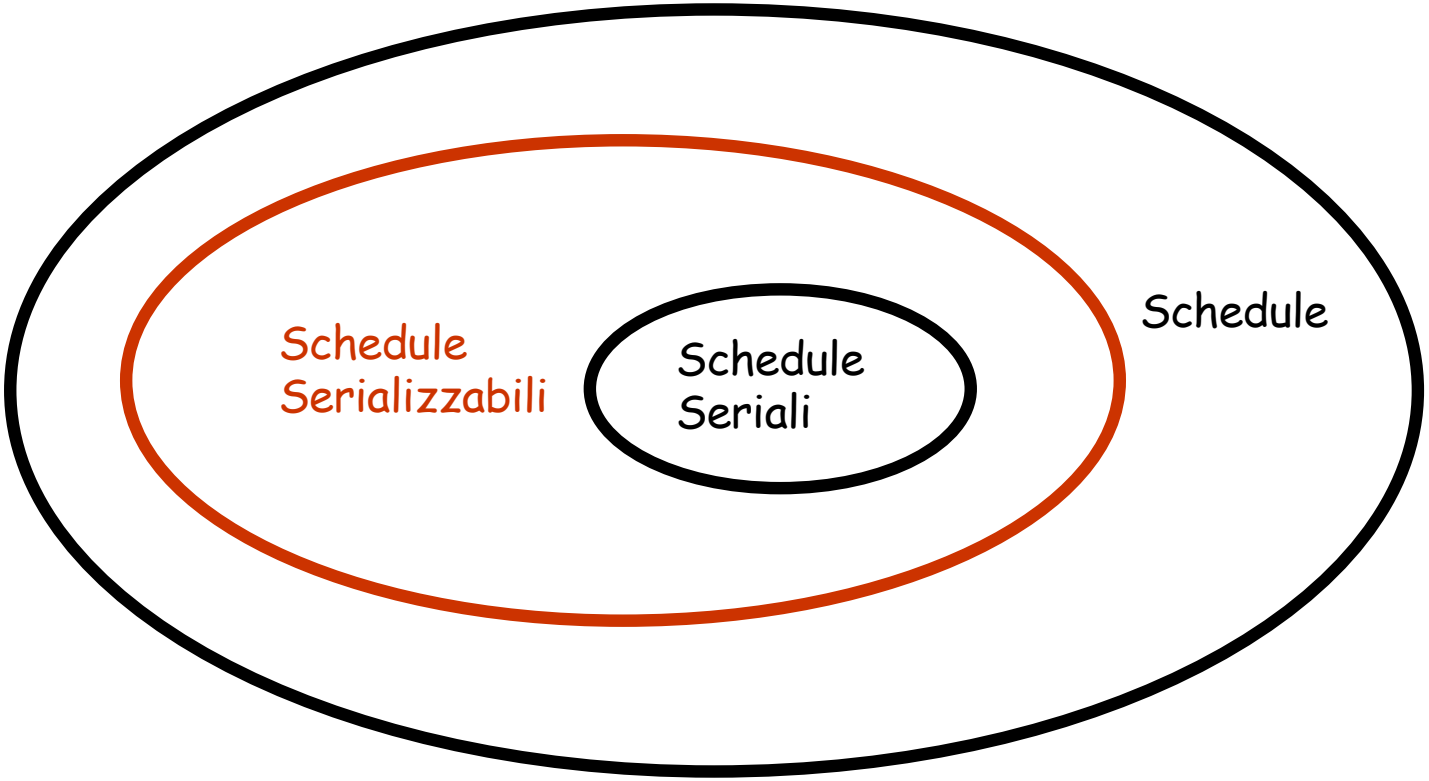
Controllo di concorrenza

- *Obiettivo*: evitare le anomalie
- *Schedule seriale:*
  - le transazioni sono separate, una alla volta

    $S_2 : r_0(x)\ r_0(y)\ w_0(x)\ r_1(y)\ r_1(x)\ w_1(y)\ r_2(x)\ r_2(y)\ r_2(z)\ w_2(z)$
  - uno schedule seriale non presenta anomalie
- *Schedule serializzabile:*
  - produce lo "stesso risultato" di uno schedule seriale sulle stesse transazioni e quindi è accettabile
    - richiede una nozione di equivalenza fra schedule
- *Controllore di concorrenza (Scheduler)*:
  - un sistema che accetta o rifiuta (o riordina) le operazioni richieste dalle transazioni (mantenendo l'ordine delle azioni in ciascuna transazione)
  - **ammettendo solo schedule serializzabili**

Idea base

- Individuare classi di schedule serializzabili
  - Se possibile verificando la serializzabilità velocemente!



View-Serializzabilità, preliminari

- Definizioni preliminari:
  - $w_i(x)$ in uno schedule S è ***scrittura finale*** *se* è l'ultima scrittura dell'oggetto x in S
  - $r_i(x)$ ***legge-da*** $w_j(x)$ in uno schedule S se il valore letto da $r_i(x)$ è quello scritto da $w_j(x)$; cioè se $w_j(x)$ precede $r_i(x)$ in S e non c'è $w_k(x)$ fra $w_j(x)$ e $r_i(x)$ in S

Legge-da e scritture finali: esempi

S_2 : $r_2(x)\ w_0(x)\ r_1(x)\ w_2(x)\ w_2(z)$

$r_1(x)$ legge da $w_0(x)$

scritture finali $w_2(x)\ w_2(z)$

S_3 : $w_0(x)\ r_2(x)\ r_1(x)\ w_2(x)\ w_2(z)$

$r_2(x)$ legge da $w_0(x)$

$r_1(x)$ legge da $w_0(x)$

scritture finali $w_2(x)\ w_2(z)$

View-Serializzabilità

- Definizioni preliminari:
  - $w_i(x)$ in uno schedule S è ***scrittura finale*** *se* è l'ultima scrittura dell'oggetto x in S
  - $r_i(x)$ ***legge-da*** $w_j(x)$ in uno schedule S se il valore letto da $r_i(x)$ è quello scritto da $w_j(x)$; cioè se $w_j(x)$ precede $r_i(x)$ in S e non c'è $w_k(x)$ fra $w_j(x)$ e $r_i(x)$ in S
- Schedule S_i e S_j(sulle stesse transazioni, con le azioni che mantengono l'ordine) sono ***view-equivalenti*** ($S_i \approx_V S_j$) se
  - stesse *scritture finali*
  - stessa relazione *legge-da*

View-Serializzabilità

- Definizioni preliminari:
  - $w_i(x)$ in uno schedule S è ***scrittura finale*** *se* è l'ultima scrittura dell'oggetto x in S
  - $r_i(x)$ ***legge-da*** $w_j(x)$ in uno schedule S se il valore letto da $r_i(x)$ è quello scritto da $w_j(x)$; cioè se $w_j(x)$ precede $r_i(x)$ in S e non c'è $w_k(x)$ fra $w_j(x)$ e $r_i(x)$ in S
- Schedule S_i e S_j(sulle stesse transazioni, con le azioni che mantengono l'ordine) sono ***view-equivalenti*** ($S_i \approx_V S_j$) se
  - stesse *scritture finali* **(stesso stato della base di dati)**
  - stessa relazione *legge-da* **(stessa "influenza" su altre transazioni)**
- Uno schedule è ***view-serializzabile*** se esiste uno schedule seriale view-equivalente ad esso
- **VSR:** l'insieme degli schedule view-serializzabili

View serializzabilità: esempi

S_2 : $r_2(x)\ w_0(x)\ r_1(x)\ w_2(x)\ w_2(z)$

S_3 : $w_0(x)\ r_2(x)\ r_1(x)\ w_2(x)\ w_2(z)$

– sono view-equivalenti?

NO

S_2 :

$r_1(x)$ legge da $w_0(x)$ scritture finali $w_2(x)\ w_2(z)$

S_3 :

$r_2(x)$ legge da $w_0(x)$ scritture finali $w_2(x)\ w_2(z)$

$r_1(x)$ legge da $w_0(x)$

View serializzabilità: esempi

S_2 : $r_2(x)\ w_0(x)\ r_1(x)\ w_2(x)\ w_2(z)$

S_3 : $w_0(x)\ r_2(x)\ r_1(x)\ w_2(x)\ w_2(z)$

- sono view-equivalenti? NO

S_4 : $w_0(x)\ r_1(x)\ r_2(x)\ w_2(x)\ w_2(z)$

- confrontiamo con S_2 e S_3

View serializzabilità: esempi

S_2 : $r_2(x)\ w_0(x)\ r_1(x)\ w_2(x)\ w_2(z)$

S_3 : $w_0(x)\ r_2(x)\ r_1(x)\ w_2(x)\ w_2(z)$

- sono view-equivalenti? NO

S_4 : $w_0(x)\ r_1(x)\ r_2(x)\ w_2(x)\ w_2(z)$

- S_4 è view-equivalente a S_3
- S_4 è seriale e quindi S_3 è view-serializzabile
- S_2 non è view-serializzabile (vediamo i vari casi)

View serializzabilità: esempi

S_2 : $r_2(x)\ w_0(x)\ r_1(x)\ w_2(x)\ w_2(z)$

S_3 : $w_0(x)\ r_2(x)\ r_1(x)\ w_2(x)\ w_2(z)$

- sono view-equivalenti?

S_4 : $w_0(x)\ r_1(x)\ r_2(x)\ w_2(x)\ w_2(z)$

- S_3 è view-equivalente a S_4
- S_4 è seriale e quindi S_3 è view-serializzabile
- S_2 non è view-serializzabile

S_5 : $w_0(x)\ r_1(x)\ w_1(x)\ r_2(x)\ w_1(z)$

S_6 : $w_0(x)\ r_1(x)\ w_1(x)\ w_1(z)\ r_2(x)$

- S_5 è view-equivalente a S_6
- S_6 è seriale e quindi S_5 è view-serializzabile

Perdita di aggiornamento (lost update)

- t_1 : r(x), x = x -200 , w(x)
- t_2 : r(x), x = x -100, w(x)

- Inizialmente x=1000; dopo esecuzione seriale (prima tutta t_1 poi tutta t_2) x=700
- Abbiamo visto un'esecuzione concorrente:

t_1	t_2
$r_1(x)$	
$x = x - 200$	
	$r_2(x)$
	$x = x - 100$
$w_1(x)$	
`commit`	
	$w_2(x)$
	`commit`

- Un aggiornamento viene perso: alla fine x=900

View serializzabilità: esempi (2)

- perdita di aggiornamento

  $S_7 : r_1(x)\ r_2(x)\ w_1(x)\ w_2(x)$

- S_7 è view-serializzabile?
  - NO, perché ha la legge-da vuota, mentre ciascuno dei due schedule seriali ha una coppia nella legge-da

Letture inconsistenti (inconsistent read)

- t_1 legge due volte x :

t_1	t_2
$r_1(x)$	
	$r_2(x)$
	$x = x + 1$
	$w_2(x)$
	`commit`
$r_1(x)$	

- t_1 legge due valori diversi per x !

View serializzabilità: esempi (2)

- perdita di aggiornamento

  $S_7 : r_1(x)\ r_2(x)\ w_1(x)\ w_2(x)$
- letture inconsistenti

  $S_8 : r_1(x)\ r_2(x)\ w_2(x)\ r_1(x)$

- S_8 è view-serializzabile?
  - NO

Aggiornamento fantasma (ghost update), lettura inconsistente (con la base di dati)

- y=1000 e z= 1000
- quindi $y + z = 2000$

t_1	t_2
$r_1(y)$	
	$r_2(y)$
	$y = y - 500$
	$r_2(z)$
	$z = z + 500$
	$w_2(y)$
	$w_2(z)$
	`commit`
$r_1(z)$	
$s = y + z$	

- $s = 2500$: ma la somma $y + z$ non è mai stata "davvero" 2500:
  - t_1 vede uno stato non esistente (o non coerente)

View serializzabilità: esempi (2)

- perdita di aggiornamento

  $S_7 : r_1(x)\ r_2(x)\ w_1(x)\ w_2(x)$
- letture inconsistenti

  $S_8 : r_1(x)\ r_2(x)\ w_2(x)\ r_1(x)$
- aggiornamento fantasma

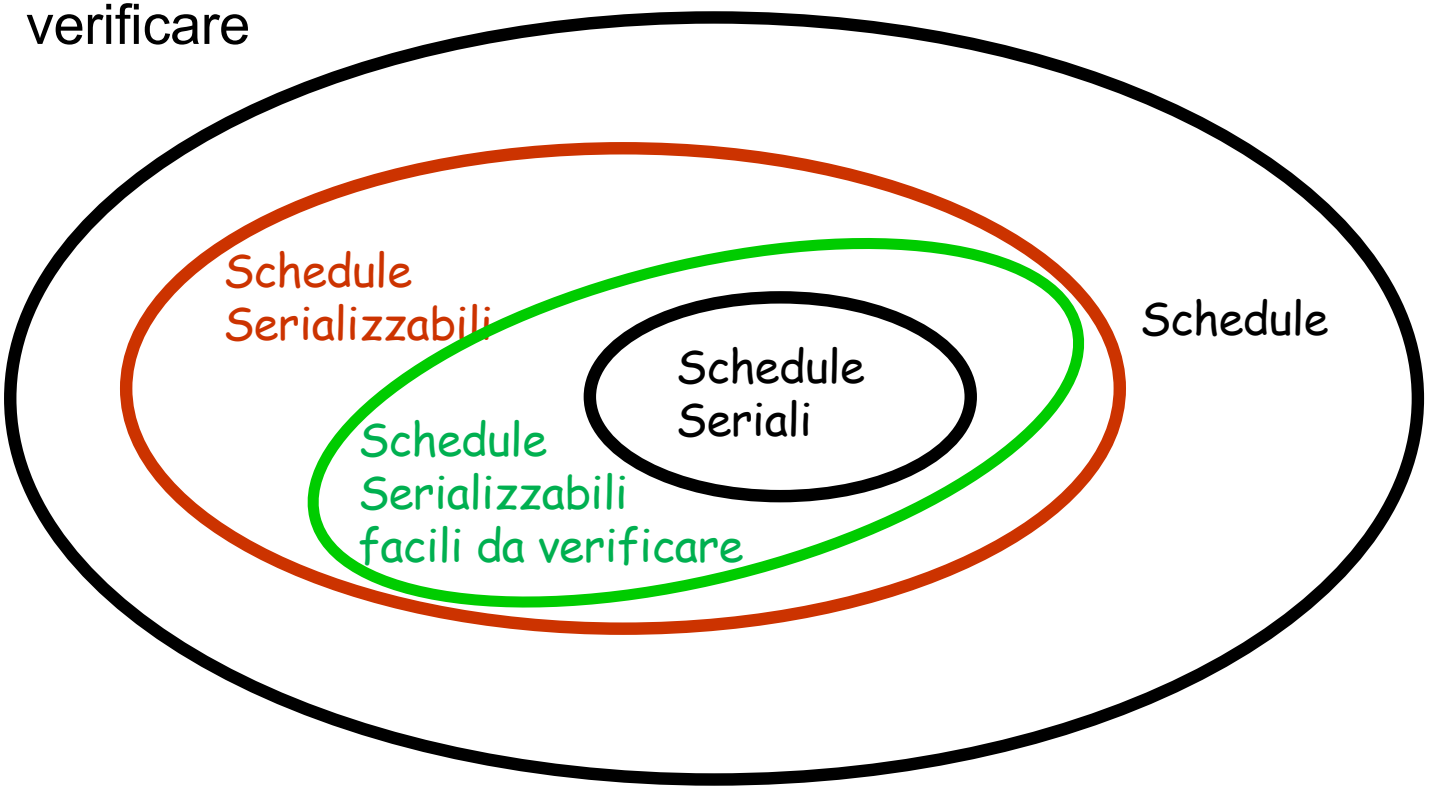
  $S_9 : r_1(y)\ r_2(z)\ r_2(y)\ w_2(y)\ w_2(z)\ r_1(z)$

- S_9 è view-serializzabile?
  - NO

View serializzabilità

- Complessità:
  - la verifica della view-equivalenza di due schedule dati:
    - polinomiale
  - decidere sulla view-serializzabilità di uno schedule:
    - problema NP-completo
- Non è utilizzabile in pratica

- Trovare tutti e soli gli schedule serializzabili è troppo costoso!
- Che facciamo?
  - Cerchiamo di trovare un loro sottoinsieme più facile da verificare



Idea fondamentale

- Proprietà più restrittive dalla view-serializzabilità (condizioni sufficienti)
  - ma più facili da verificare
- Quindi, "rinunceremo" ad alcuni schedule serializzabili, ma sapremo decidere rapidamente

- Percorso:
  - Un'altra proprietà "teorica":
    - conflict-serializzabilità
  - Tecniche utilizzabili in pratica
    - 2PL
    - multiversione

Conflict-serializzabilità: preliminari

- Definizione propedeutica:
  - Un'azione a_i è in *conflitto* con un'altra a_j
    - $i \neq j$, cioè sono azioni di transazioni diverse
    - a_i e a_j operano sullo stesso oggetto
    - almeno una di esse è una scrittura.
  - Due casi:
    - conflitto *read-write* (*rw* o *wr*)
    - conflitto *write-write* (*ww*).

Conflict-serializzabilità

- Definizione propedeutica:
  - un'azione a_i è in *conflitto* con un'altra a_j
    - $i \neq j$, cioè sono azioni di transazioni diverse
    - a_i e a_j operano sullo stesso oggetto
    - almeno una di esse è una scrittura.

Esempi

- Troviamo i conflitti per i due schedule seguenti
  - $r_1(x)$, $r_2(x)$, $r_3(y)$, $w_3(y)$, $r_1(y)$, $w_2(x)$, $w_1(x)$
  - $r_1(x)$, $w_1(x)$, $r_3(y)$, $w_3(y)$, $r_1(y)$, $r_2(x)$, $w_2(x)$

  - Alla lavagna!

Esempi

- Troviamo i conflitti per i due schedule seguenti
  - $r_1(x), r_2(x), r_3(y), w_3(y), r_1(y), w_2(x), w_1(x)$
  - $r_1(x), w_1(x), r_3(y), w_3(y), r_1(y), r_2(x), w_2(x)$

Conflict-serializzabilità

- Definizione propedeutica:
  - un'azione a_i è in *conflitto* con un'altra a_j
    - $i \neq j$, cioè sono azioni di transazioni diverse
    - a_i e a_j operano sullo stesso oggetto
    - almeno una di esse è una scrittura.
- Schedule S_i e S_j (sulle stesse transazioni, con le azioni che mantengono l'ordine) sono *conflict-equivalenti* ($S_i \approx_C S_j$) se:
  - ogni coppia di operazioni in conflitto ha nei due schedule il medesimo ordine
- Uno schedule è *conflict-serializable* se esiste uno schedule seriale conflict-equivalente ad esso
- **CSR:** l'insieme degli schedule conflict-serializzabili

CSR e VSR

Teorema: Ogni schedule conflict-serializzabile è view-serializzabile, ma non necessariamente viceversa

- Controesempio per la non necessità:

  $r_1(x)\ w_2(x)\ w_1(x)\ w_3(x)$

  scritture finali

  – view-serializzabile:
    - view-equivalente a $r_1(x)\ w_1(x)\ w_2(x)\ w_3(x)$

legge da vuoto

CSR e VSR

Teorema: Ogni schedule conflict-serializzabile è view-serializzabile, ma non necessariamente viceversa

- Controesempio per la non necessità:

  $r_1(x)\ w_2(x)\ w_1(x)\ w_3(x)$

  – view-serializzabile:
    - view-equivalente a $r_1(x)\ w_1(x)\ w_2(x)\ w_3(x)$

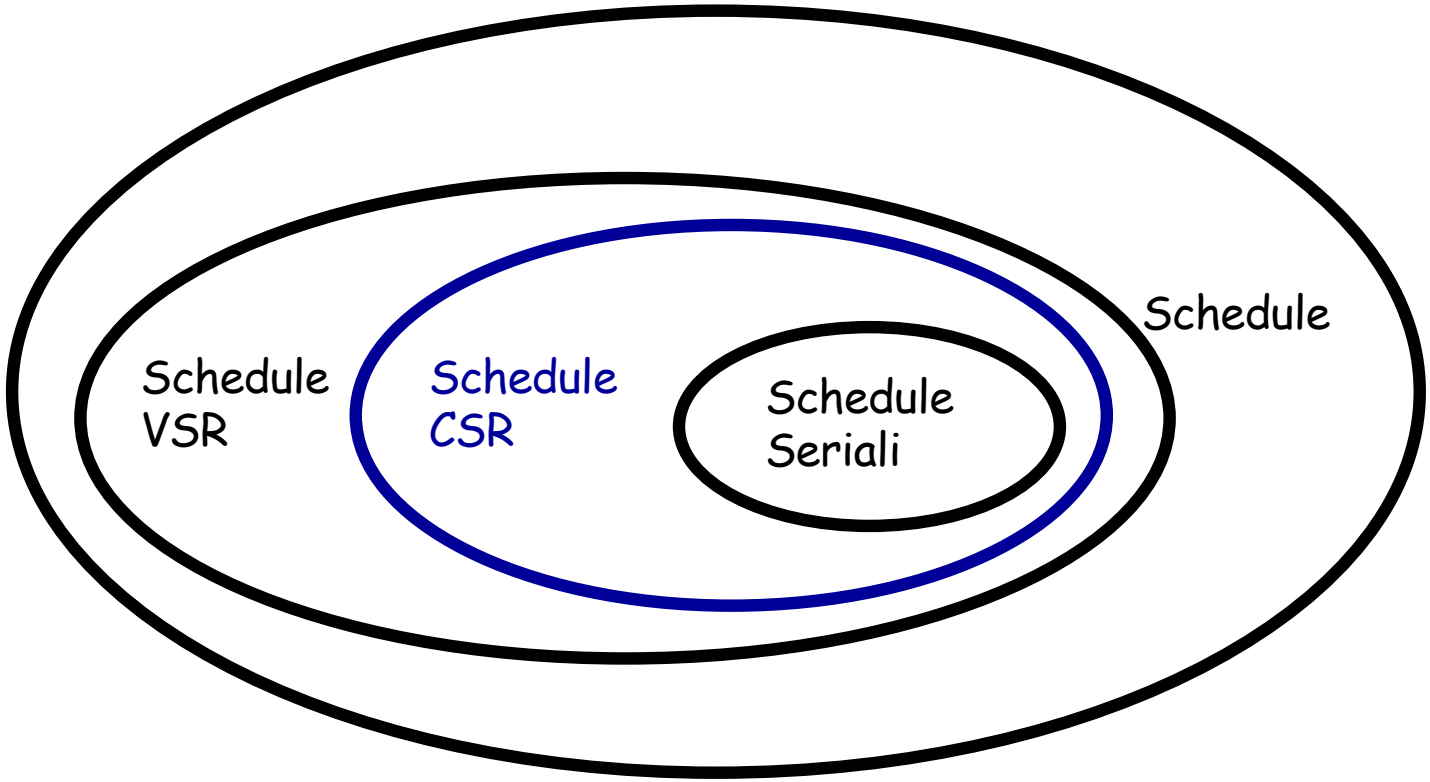
  – non conflict-serializzabile
    - conflitti $r_1(x)\ w_2(x)$, $w_2(x)\ w_1(x)$

- Sufficienza: vediamo

CSR implica VSR

- CSR: esiste schedule seriale conflict-equivalente
- VSR: esiste schedule seriale view-equivalente
- Per dimostrare che CSR implica VSR è sufficiente dimostrare che la conflict-equivalenza $\approx_C$ implica la view-equivalenza $\approx_V$, cioè che se due schedule sono $\approx_C$ allora sono $\approx_V$
- Supponiamo $S_1 \approx_C S_2$ e dimostriamo che $S_1 \approx_V S_2$.
  - stesse scritture finali:
    - se così non fosse, ci sarebbero due scritture (su uno stesso oggetto) in ordine diverso e poiché due scritture (su uno stesso oggetto) sono in conflitto i due schedule non sarebbero $\approx_C$
  - stessa relazione "legge-da":
    - se così non fosse, ci sarebbero scritture (su uno stesso oggetto) in ordine diverso o coppie lettura-scrittura (su …) in ordine diverso e quindi, come sopra, sarebbe violata la $\approx_C$

CSR e VSR



Verifica di conflict-serializzabilità

- Per mezzo del **grafo dei conflitti** (grafo orientato)
  - un nodo per ogni transazione t_i
  - un arco (orientato) da t_i a t_j se c'è almeno un conflitto fra un'azione a_i e un'azione a_j tale che a_i precede a_j

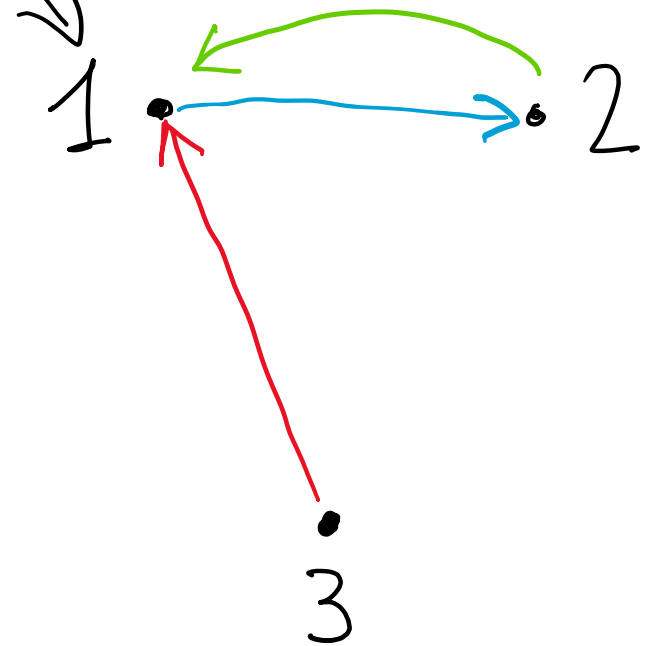
Esempi

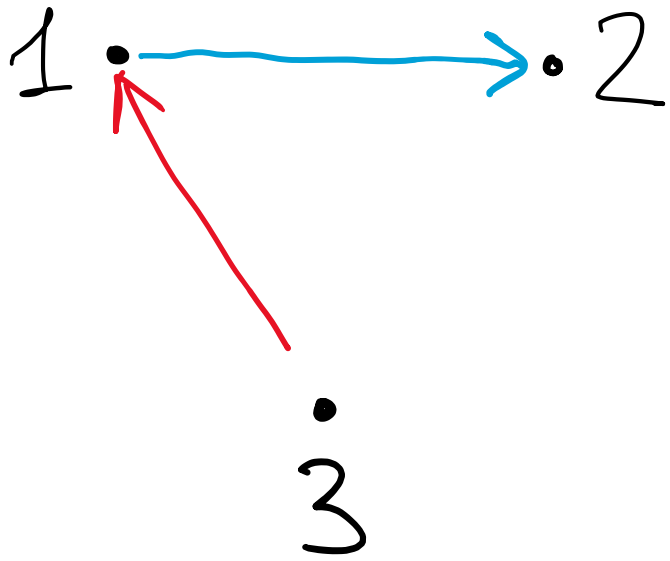
- Costruire il grafo dei conflitti per i due schedule seguenti
  - $r_1(x)$, $r_2(x)$, $r_3(y)$, $w_3(y)$, $r_1(y)$, $w_2(x)$, $w_1(x)$
  - $r_1(x)$, $w_1(x)$, $r_3(y)$, $w_3(y)$, $r_1(y)$, $r_2(x)$, $w_2(x)$

  - Alla lavagna!

Esempi

- Costruire il grafo dei conflitti per i due schedule seguenti
  - $r_1(x), r_2(x), r_3(y), w_3(y), r_1(y), w_2(x), w_1(x)$
  - $r_1(x), w_1(x), r_3(y), w_3(y), r_1(y), r_2(x), w_2(x)$





CSR e aciclicità del grafo dei conflitti

Teorema **S è in CSR se e solo se il grafo di S è aciclico**

Grafo dei conflitti e conflict-equivalenza

Lemma **Due schedule conflict-equivalenti hanno lo stesso grafo dei conflitti**

- **$S_1 \approx_C S_2$;** dimostriamo che **S_1e S_2 hanno lo stesso grafo.**
  - Per assurdo: supponiamo che S_1 e S_2 siano equivalenti e non abbiano lo stesso grafo.
    Allora nel grafo di uno dei due schedule c'è un arco non presente nell'altro; supponiamo che S_1 abbia un tale arco; esso è relativo a due azioni che in S_1 sono in conflitto;
    se l'arco non compare nel grafo di S_2, allora le due azioni sono in S_2 ordine diverso e quindi S_1 e S_2 non sono $\approx_C$: una contraddizione

CSR e aciclicità del grafo dei conflitti

Teorema **S è in CSR se e solo se il grafo di S è aciclico**

- **Se S è in CSR allora il grafo di S è aciclico**

  Se S è in CSR allora è $\approx_C$ ad uno schedule seriale S_0.

  Per il lemma, S e S_0 hanno lo stesso grafo

  Il grafo di S_0, che è seriale, è aciclico, perché ci possono essere conflitti fra azioni a_i e a_j solo se $i<j$ (mentre un ciclo richiede almeno un arco (i,j) con $i>j$).

  Quindi il grafo di S è aciclico.

- **Se il grafo di S è aciclico allora S è in CSR**

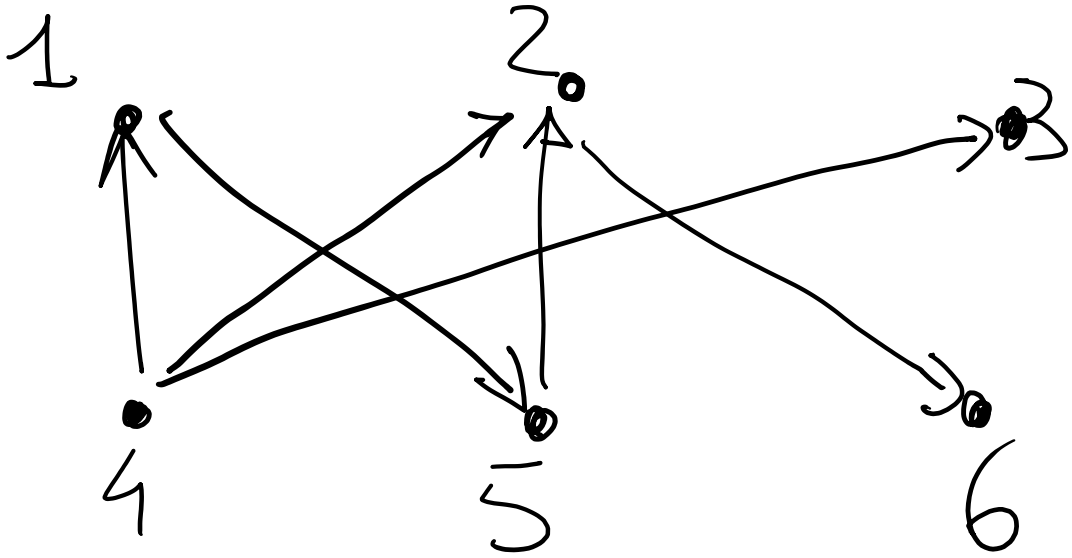
  Se il grafo è aciclico, allora esiste fra i nodi un "ordinamento topologico" (cioè una numerazione dei nodi tale che il grafo contiene solo archi (i,j) con $i<j$).

  Lo schedule seriale le cui transazioni sono ordinate secondo l'ordinamento topologico è equivalente a S, perché per tutti i conflitti (i,j) si ha sempre $i<j$.

  Quindi esiste uno schedule seriale equivalente ad S: S è in CSR

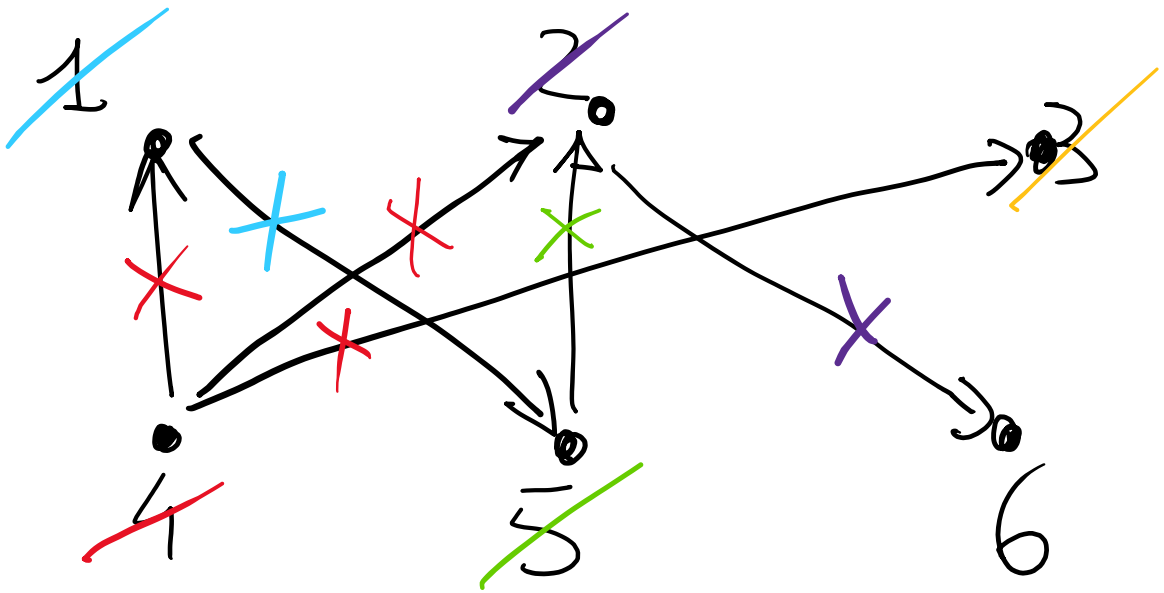
Ordinamento topologico

- Troviamo l'ordinamento topologico per questo grafo

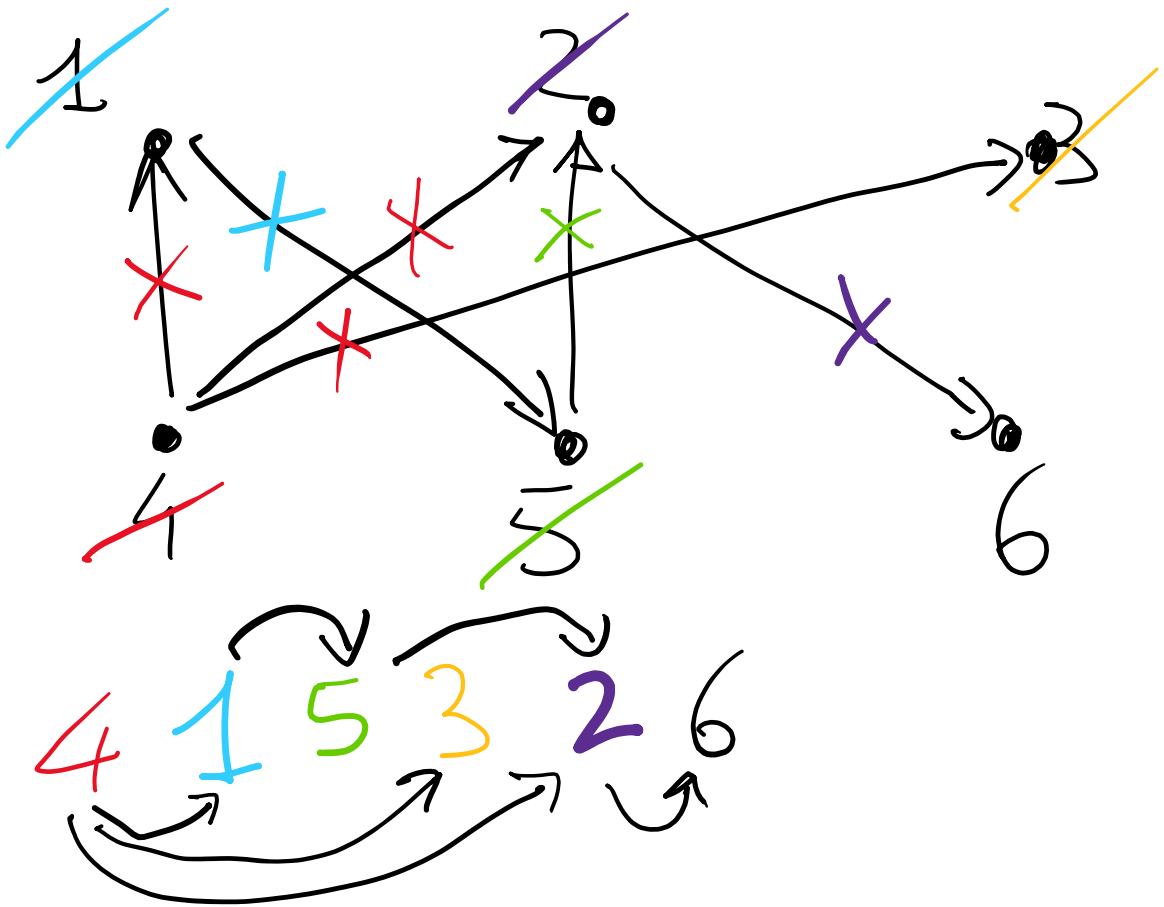


Alla lavagna!

Ordinamento topologico



4 1 5 3 2 6


1
2
3
4
5
6
4 1 5 3 2 6

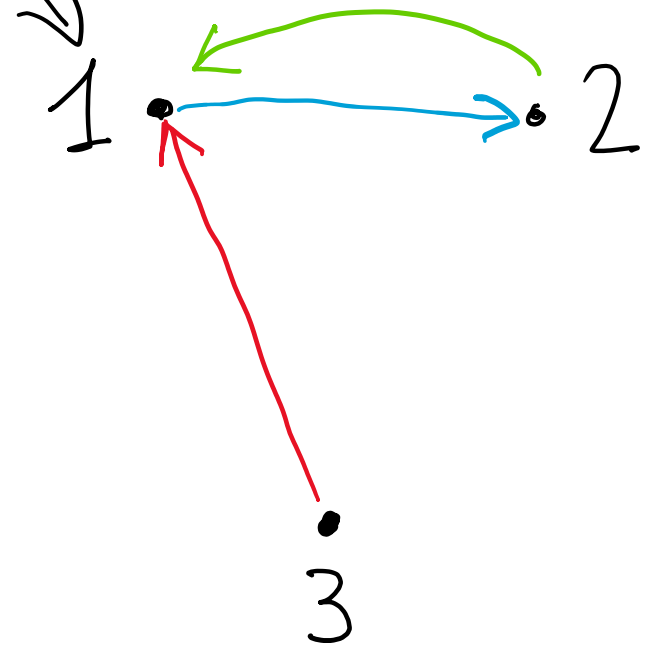
Ancora gli esempi alla lavagna

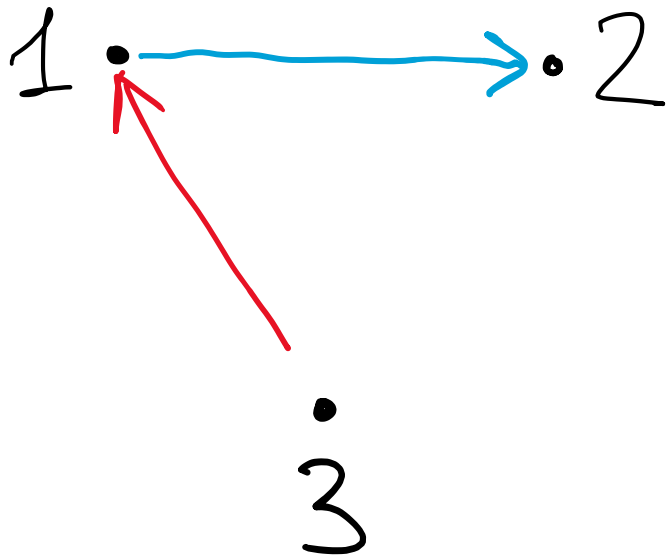
- Sono conflict-serializzabili?
  - $r_1(x)$, $r_2(x)$, $r_3(y)$, $w_3(y)$, $r_1(y)$, $w_2(x)$, $w_1(x)$
  - $r_1(x)$, $w_1(x)$, $r_3(y)$, $w_3(y)$, $r_1(y)$, $r_2(x)$, $w_2(x)$

    (torniamo alla lavagna – lucido 55)

Esempi

- Costruire il grafo dei conflitti per i due schedule seguenti
  - $r_1(x), r_2(x), r_3(y), w_3(y), r_1(y), w_2(x), w_1(x)$
  - $r_1(x), w_1(x), r_3(y), w_3(y), r_1(y), r_2(x), w_2(x)$





Controllo della concorrenza in pratica

- La conflict-serializabilità
  - è verificabile più rapidamente della view-serializzabilità (con opportune strutture dati, complessità lineare),
- Però
  - sarebbe davvero efficiente se potessimo conoscere il grafo dall'inizio, ma così non è:
  - uno scheduler deve operare “incrementalmente”, cioè ad ogni richiesta di operazione decidere se eseguirla subito oppure fare qualcos'altro;
  - non è praticabile mantenere il grafo, aggiornarlo e verificarne l'aciclicità ad ogni richiesta di operazione
  - si basa sull'ipotesi di commit-proiezione
- Quindi, anche la conflict-serializzabilità
  - è inutilizzabile in pratica in modo diretto

Controllo della concorrenza in pratica, 2

- In pratica, si utilizzano tecniche che
  - garantiscono la conflict-serializzabilità senza dover costruire il grafo
  - non richiedono l'ipotesi della commit-proiezione

- Le tecniche utilizzate
  - 2PL (two-phase locking)
  - timestamp (implementazione "multiversion")